SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO3	**Assessment:**	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	**40%**	**Total Marks:**	**25**
CO3 Statement:	*Design processor organization by constructing pipelined Datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)*		
Student Name:	N. Tharun Sai	**Reg.No.:**	192472382
Department:	CSE-AI	**Date:**	

Code of Conduct:

I N. Tharun Sai (192472382) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

N. Tharun Sai

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyse the sequence of operations and the role of the ALU in processing data.

2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.

3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.

4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.

5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental	30	Well-planned, systematic implementation with	Correct	Basic	Incorrect or incomplete

Design		optimal solution	implementation with minor issues	implementation with errors or inefficiencies	implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect an alysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1. Register Transfer Operations and ALU Operations

Aim

To design and simulate register transfer operations along with basic arithmetic and logic operations such as **Addition, Subtraction, AND, and OR**, using multiple registers and an ALU.

Algorithm

2. Initialize multiple registers R1, R2, R3, and R4.
3. Store input values in R1 and R2.
4. Transfer the contents of R1 to R3.
5. Transfer the contents of R2 to R4.
6. Perform addition of R3 and R4 using the ALU.
7. Perform subtraction of R3 and R4.
8. Perform AND and OR operations.
9. Store and display the results.
10. Analyze the sequence of register transfers and ALU operations.

Code

Result

Thus, register transfer operations and basic ALU operations were successfully simulated. The ALU performs arithmetic and logical operations on data received from the registers.

2. Single-Bus and Multi-Bus Organization

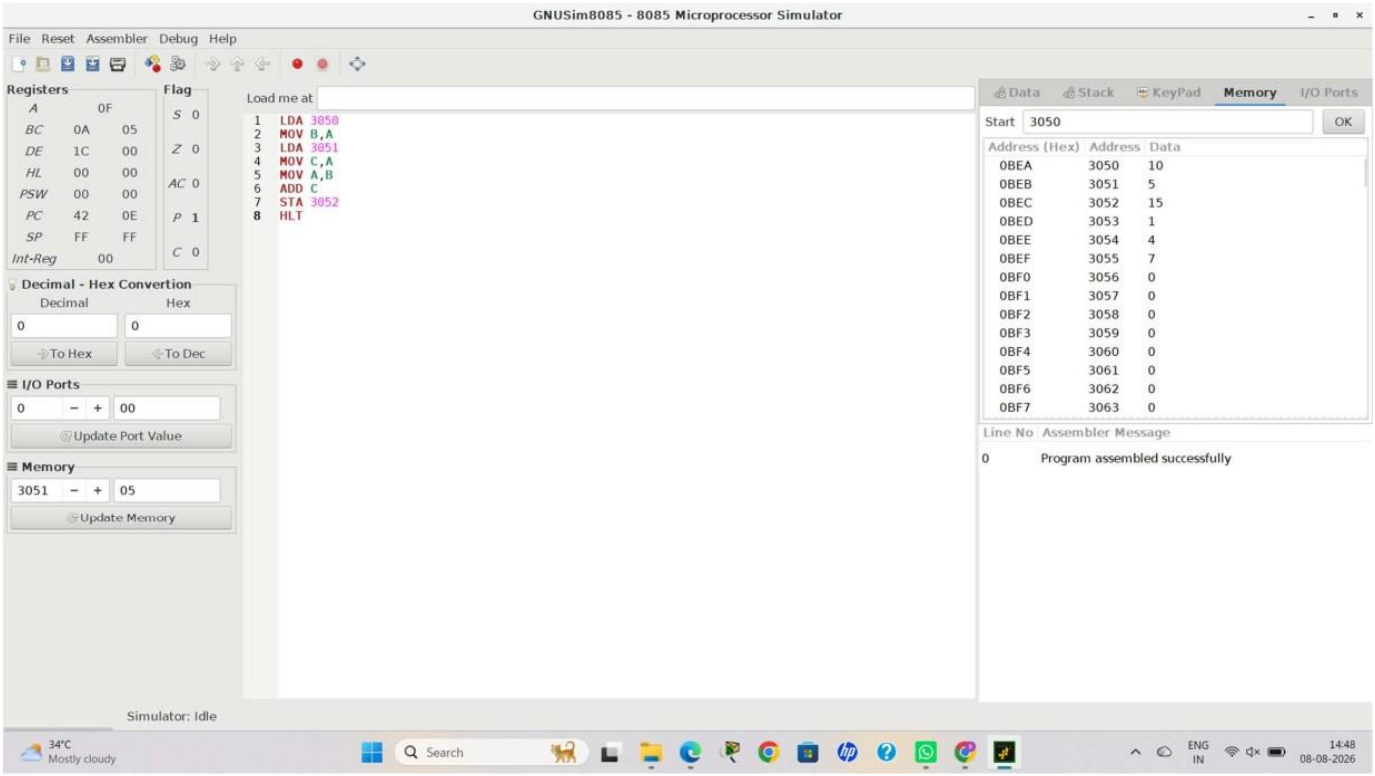
Aim

To model and compare **single-bus and multi-bus computer organization** by performing data transfers between registers, memory, and I/O, and analyse their execution time and efficiency.

Algorithm

1. Create registers R1, R2, and R3.
2. Create memory and I/O locations.
3. In the single-bus system, perform transfers sequentially.
4. Count the number of cycles required.
5. In the multi-bus system, allow independent transfers simultaneously.
6. Count the cycles required.
7. Calculate the speedup.
8. Compare both architectures.

Code



Result

The multi-bus organization completed the same data transfers in fewer cycles. It improves parallel data transfer and reduces the bottleneck associated with a single shared bus.

3. Five-Stage Instruction Pipeline

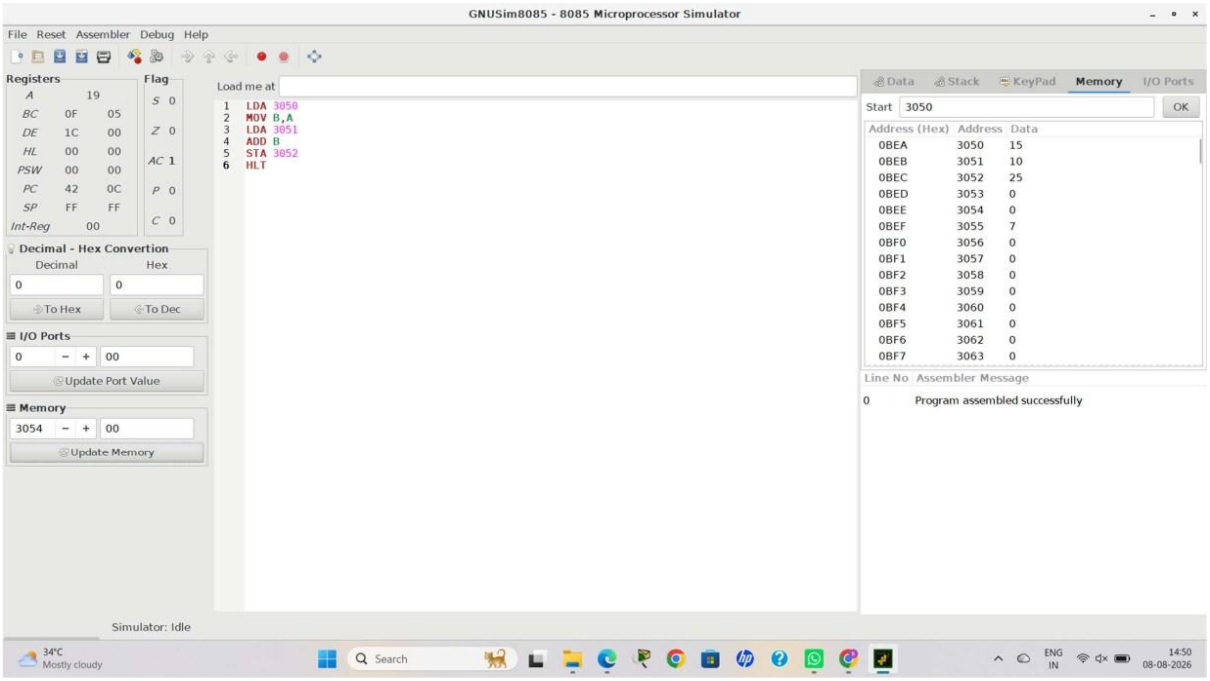
Aim

To simulate a **5-stage instruction pipeline** consisting of Fetch, Decode, Execute, Memory, and Write-back stages and compare its performance with a non-pipelined processor.

Algorithm

1. Define the five pipeline stages:
    a. IF – Instruction Fetch
    b. ID – Instruction Decode
    c. EX – Execute
    d. MEM – Memory Access
    e. WB – Write Back
2. Define a set of instructions.
3. Execute the instructions sequentially in a non-pipelined processor.
4. Calculate non-pipelined execution time.
5. Execute the same instructions using pipeline stages.
6. Calculate pipelined execution time.
7. Calculate speedup and throughput.
8. Compare both systems.

Code



Result

The 5-stage pipeline successfully executed multiple instructions simultaneously. Compared with the non-pipelined processor, pipelining reduced the total execution time and increased instruction throughput.

4. Pipeline Hazards and Hazard Resolution

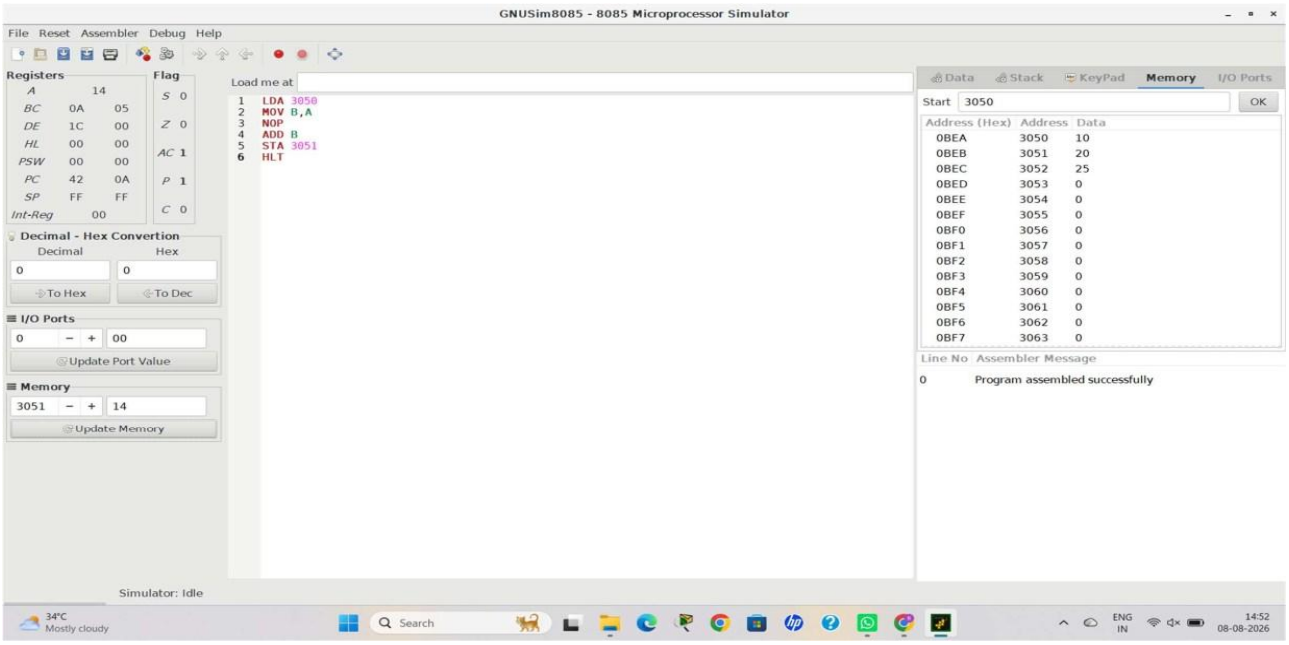
Aim

To develop a pipeline execution scenario containing **data, control, and structural hazards**, and study the effects of stalling, data forwarding, and branch prediction on processor performance.

Algorithm

1. Define a sequence of dependent instructions.
2. Identify data hazards caused by dependencies between instructions.
3. Identify a control hazard caused by a branch instruction.
4. Identify a structural hazard caused by competing resource requirements.
5. Calculate execution cycles without hazard handling.
6. Apply stalling to resolve hazards.
7. Apply data forwarding to reduce data stalls.
8. Apply branch prediction to reduce control stalls.
9. Compare the execution cycles

Code



Result

The experiment demonstrated that pipeline hazards can increase execution time. **Data forwarding** reduces data hazards, while **branch prediction** reduces control-hazard penalties. These techniques improve pipeline efficiency and reduce the number of stall cycles.

5. Superscalar, Out-of-Order, Speculative Execution and SMT

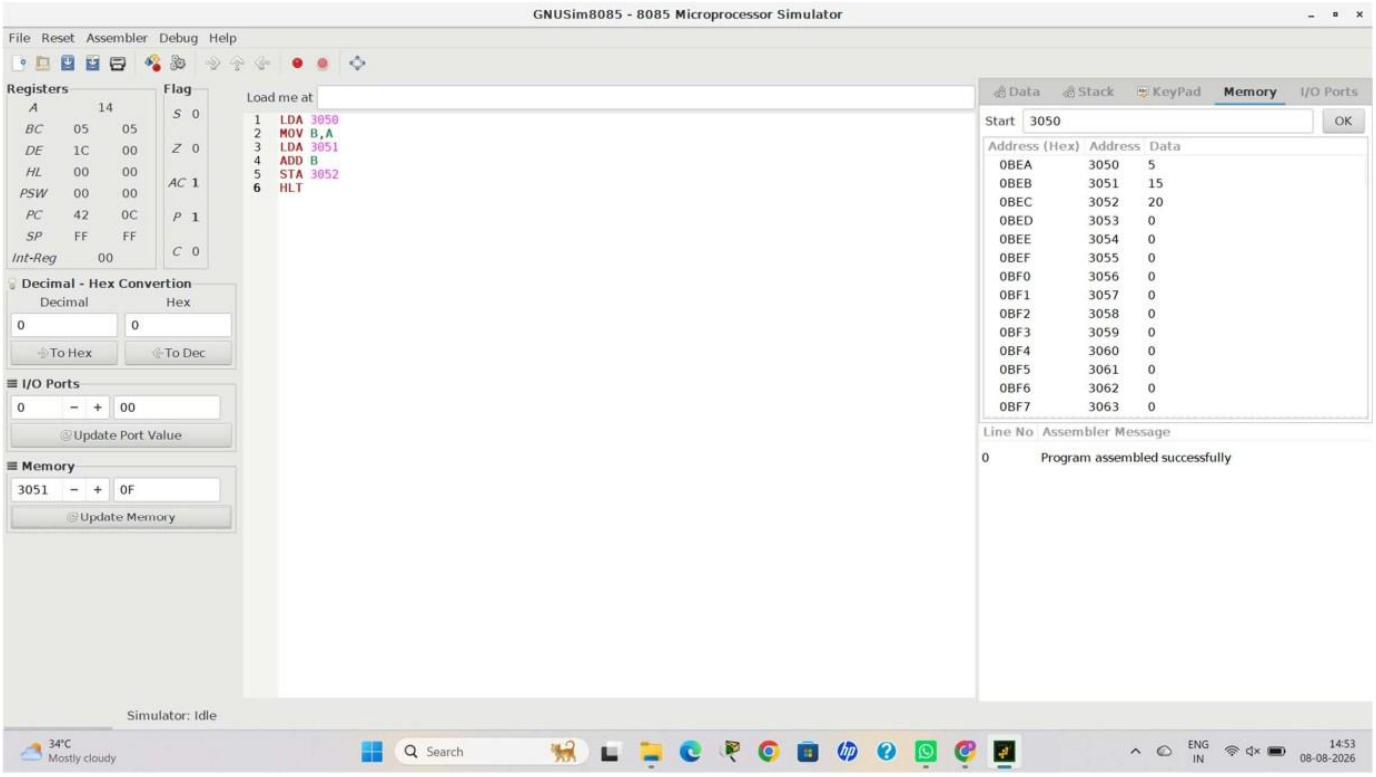
Aim

To simulate a processor supporting **superscalar, out-of-order, speculative execution, and simultaneous multithreading (SMT)** and evaluate improvements in instruction throughput and resource utilization.

Algorithm

1. Define a set of independent instructions.
2. Execute instructions sequentially using a single-issue processor.
3. Execute two instructions per cycle using a superscalar processor.
4. Allow instructions to execute out of order when resources are available.
5. Simulate speculative execution for branch-dependent instructions.
6. Introduce two threads to represent SMT.
7. Execute instructions from both threads in parallel.
8. Calculate execution cycles, throughput, and resource utilization.
9. Compare single-issue, superscalar, and SMT approaches.

Code



Result

The experiment demonstrated the advantages of advanced processor architectures. **Superscalar execution** increases the number of instructions issued per cycle, **out-of-order execution** uses processor resources more efficiently, **speculative execution** reduces branch delays, and **SMT** allows instructions from multiple threads to use processor resources simultaneously. These techniques improve overall instruction throughput and resource utilization.